

qml by klnsss for msrit Beng 2826 day1 (part2)

for Msrit by sarma

2nd August 2026

here the agenda is to express, recap of

- 3 different bases of quantum computing
- CNOT gate(2 qubit), Hadamard gate(1 qubit), Rotation gates (1 qubit)
- Phase kick
- Deutsch algo
- QPE (quantum phase estimation, also QFT)
- Relevant number theory results for Shor's Algo.
- Grover's Algo.
- Quantum speedup

3 different orthonormal bases of quantum computing

- Z-basis, X-basis (Hadamard basis / diagonal basis), Y-basis (circular basis)
- Z-basis $\mapsto$ called Energy basis (correspond to qubit energy levels)
- $\mapsto$ computational basis / standard basis / Z - basis states
- $|0\rangle \mapsto$ top of Bloch sphere
- $|1\rangle \mapsto$ opposite to $|0\rangle$
- $|0\rangle = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, |1\rangle = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$

- state vectors are
- $|+\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \end{bmatrix}$
- $|-\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle) = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ -1 \end{bmatrix}$
- associated observable is —pauli X matrix
$$X = |+\rangle \langle +| (-) |-\rangle \langle -| = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$$

- state vectors are
- $|+i\rangle = \frac{1}{\sqrt{2}}(|0\rangle + i|1\rangle) = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ i \end{bmatrix}$
- $|-i\rangle = \frac{1}{\sqrt{2}}(|0\rangle - i|1\rangle) = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ -i \end{bmatrix}$
- associated observable is —pauli Y matrix
$$Y = |+i\rangle \langle +i| (-) |-i\rangle \langle -i| = \begin{bmatrix} 0 & -i \\ i & 0 \end{bmatrix}$$

- $|U\rangle = |+\rangle \langle 0| (+) |-\rangle \langle 1|$
- $\Downarrow$
- $|+\rangle = U|0\rangle \implies |+\rangle \langle 0| = U|0\rangle \langle 0|$
- $|-\rangle = U|1\rangle \implies |-\rangle \langle 1| = U|1\rangle \langle 1|$

- used to create superposition, since, it $\xrightarrow{\text{transforms}}$ $|0\rangle \mapsto |+\rangle$,
and $|1\rangle \mapsto |-\rangle$
- $H^2 = I(\text{involutory})$
- $H|0\rangle = |+\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$
- $H|1\rangle = |-\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)$
- $H|+\rangle = |0\rangle$
- $H|-\rangle = |1\rangle$

- class of gates which depend on a **continuous parametre, usually an angle of rotation—parametric quantum gates**
- these gates ensure— control over qubit states (extensively used in quantum Algorithms)
- Rotation gates $\Leftrightarrow$ allow transformations around Bloch's sphere's axes
- Universal gate sets enable any quantum operation of need.
- for any arbitrary single-qubit operation, rotation gates are used to manipulate qubit states.

- $R_x(\theta) |\psi\rangle = \cos \frac{\theta}{2} |\psi\rangle - i \sin \frac{\theta}{2} X |\psi\rangle \mapsto$ when applied
- $R_x(|\theta\rangle) = [\cos \frac{\theta}{2} - i \sin \frac{\theta}{2} - i \sin \frac{\theta}{2} \cos \frac{\theta}{2}] \mapsto$ matrix
- $R_y(|\theta\rangle) = [\cos \frac{\theta}{2} - \sin \frac{\theta}{2} - \sin \frac{\theta}{2} \cos \frac{\theta}{2}] \mapsto$ matrix
- $R_y(\theta) |\psi\rangle = \cos \frac{\theta}{2} |\psi\rangle - \sin \frac{\theta}{2} Y |\psi\rangle \mapsto$ when applied
- $R_z(\theta) = \begin{bmatrix} e^{-i\frac{\theta}{2}} & 0 \\ 0 & e^{i\frac{\theta}{2}} \end{bmatrix}$
- $R_z(\theta) |\psi\rangle = e^{-\frac{\theta}{2}} |0\rangle + e^{-\frac{\theta}{2}} |1\rangle \mapsto$ when applied to a qubit state
- $R_x \mapsto$ Rotation around x-axis
- $R_y \mapsto$ Rotation around y-axis
- $R_z \mapsto$ Rotation around z-axis

- **UNiversal gates**: a small collection of gates capable of approximating any quantum operation to arbitrary precision and are fundamental for quantum computing.(look at next slide for tabular info)
- clifford gates: (H,S,CNOT),
- T gate $\mapsto$ non clifford but essential for. || phase shift gate with $\phi = \frac{\pi}{4}$ universality
- T gate matrix =
$$\begin{bmatrix} 1 & 0 \\ 0 & e^{\frac{i\pi}{4}} \end{bmatrix}$$
- useful in grover's and or Shor's factoring algorithm

universal gate set	elaboration
$\{H, T, CNOT\}$	minimal clifford+T set
$\{H, S, T, CNOT\}$	extended clifford +T
$\{R_x(\theta), R_y(\theta), CNOT\}$	parametrized for single qubit
$\{X, Y, Z, H, S, T, CNOT\}$	for simulation rep'n
Hardware basis gates	native to devices(ignore)

- WHEN CX (CNOT) gate is applied to the target value $\left(\frac{|0\rangle - |1\rangle}{\sqrt{2}}\right)$ with control qubit $|1\rangle \implies$
- $CX |1\rangle \cdot \left(\frac{|0\rangle - |1\rangle}{\sqrt{2}}\right) = |1\rangle \cdot \left((-1) \cdot \left(\frac{|0\rangle - |1\rangle}{\sqrt{2}}\right)\right) \implies$
- $CX |1\rangle \cdot \left(\frac{|0\rangle - |1\rangle}{\sqrt{2}}\right) = -|1\rangle \cdot \left(\frac{|0\rangle - |1\rangle}{\sqrt{2}}\right) \implies$
- nothing happens to control qubit $|0\rangle$, no change, $\implies$
- $CX |0\rangle \cdot \left(\frac{|0\rangle - |1\rangle}{\sqrt{2}}\right) = |0\rangle \cdot \left(\frac{|0\rangle - |1\rangle}{\sqrt{2}}\right) \Leftrightarrow \text{phase(target value) is kicked back to register}$
- CNOT (2 qubit) matrix is in next slide.

- $$\text{CNOT} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix} = \begin{bmatrix} I & 0 \\ 0 & X \end{bmatrix}$$

- $$\begin{matrix} & \langle 00| & \langle 01| & \langle 10| & \langle 11| \\ \begin{matrix} |00\rangle \\ |01\rangle \\ |10\rangle \\ |11\rangle \end{matrix} & \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix} \end{matrix}$$

- $CNOT |\psi\rangle = \alpha |00\rangle + \beta |01\rangle + \gamma |11\rangle + \delta |10\rangle$
- $CNOT |++\rangle = |++\rangle$
- $CNOT |+-\rangle = |--\rangle$
- $CNOT |-+\rangle = |-+\rangle$
- $CNOT |--\rangle = |+-\rangle$

Quantum Algorithms (QML aligned) intro

Agenda here is :

- concept of quantum speedup
- Deutsch-Jozsa algorithm (more complex and appropriate for QML than simpler 'Deutsch'(prescribed))
- Grover's algorithm
- Shor's Algorithm

- quantum speedup—potential computational benefit / advantage that quantum algorithms have over classical algorithms in solving certain types of computational problems.
- 2 types of quantum speedup
- 1)—POLYNOMIAL SPEEDUP
- 2) — exponential speedup
- polynomial speedup— if classical computation time= n^k , and. quantum computation time will be— n^b where $b < k$
- exponential speedup— exponential time for classical computation where as , only , polynomial time on a quantum computer.

time complexity notation of Algorithms

17

- $T(n)$ —classical time complexity of size 'n'
- $Q(n)$ —quantum time complexity for same 'n'
- quantum speedup $S(n) = \frac{T(n)}{Q(n)}$
- larger the $s(n)$ —greater speedup
- $S(n)$ —is exponential $\implies$ quantum algorithm is exponentially faster as compared to best known classical algorithm

- shor's algorithm for integer factorization
- shor's classical factoring— $\exp\left(\mathcal{O}((\log N)^{\frac{1}{3}}(\log\log N)^{\frac{2}{3}})\right)$
- shor's quantum version— $\mathcal{O}((\log N)^2(\log\log N)(\log\log\log N))$ is polynomially faster.
- Grover's Algorithm $\xrightarrow{\text{quadratics speedup}}$ for unstructured search problems
- Grover's classical— $\mathcal{O}(N)$ — — — *linear time to find target*
- Grover's quantum— $\mathcal{O}(\sqrt{N})$ —quadratic improvement

problem representation of quantum algorithms

- quantum algorithms reformulate classical problems into unitary operations compatible with quantum systems.
- popular QML examples– (i) optimization problems (ii) variational quantum algorithms
- quantum algorithms exploit the properties like (a) superposition (b) entanglement, (c) interference to solve computational problems more efficiently than classical algorithms in specific contexts
- quantum algorithms process information in parallel by manipulating quantum states in superposition
- quantum capability provides speedups / exponential speedups

Deutsch-Jozsa Algorithm, and few other qml problems intro

- enables quantum speedup for **Decision problems**
- Def'n of decision problems—problems for which certain/
some algorithms exist which always
halts with the correct 'yes' or 'no' answer
- Decision problems are also called Turing-decidable /
computable problems.
- VQE, QAOA solve optimization challenges.
- QNN's, QSVMs integrate quantum power into classical
models (enhanced / inspired)

- determines whether a function $f(x)$ is constant or balanced using quantum parallelism.
- $f(x)$ is constant if it outputs the same value for all inputs
- **Balanced** if it outputs an equal number of $|0\rangle$, $|1\rangle$ solutions.
- for a function of 'n' bits of input $\implies$
- with classical evaluation $\xrightarrow{\text{requires}} 2^{n-1} + 1$ queries (worst case)
- **with quantum algorithm— solved with just 1 query.**

balanced / quantum function details–tabular

22

x	$f^{00}(x)$	$f^{01}(x)$	$f^{10}(x)$	$f^{11}(x)$
0	0	0	1	1
1	0	1	0	1
	Constant	Balanced	Balanced	Constant

Figure: from a publication with thanks

simple case– direct Deutsch algo relevance

XOR function

a	b	$a \oplus b$
0	0	0
0	1	1
1	0	1
1	1	0

Figure: from a published book with thanks

another table (XOR) in D-J context

24

Property	Name
$x \oplus x = 0$	X1
$x \oplus 0 = x$	X2
$(x \oplus y) \oplus z = x \oplus (y \oplus z)$	X3

Figure: taken from a published book with thanks

4 oracles of Deutsch-Jozsa with $8(2^3)$ possibilities

x	$f_0(x)$	$f_1(x)$	$f_2(x)$	$f_3(x)$
000	0	1	0	1
001	0	1	0	1
010	0	1	0	0
011	0	1	0	1
100	0	1	1	0
101	0	1	1	0
110	0	1	1	0
111	0	1	1	1

Figure: with thanks to the publisher / author

- circuit initialization (step 1)
- starts with 2 qubits
- 1—in the superposition state for input
- $|\psi\rangle = H^{\otimes n} |\{0\}^{\otimes n} \otimes H|1\rangle$
- another (2nd qubit) as auxiliary qubit for the oracle
- oracle Application (2nd step)— to encode the function $f(x)$
- Result measurement (3rd step)— Hadamard gate applied again and output qubits are measured

- VQA's use hybrid quantum-classical approaches to solve problems like—— optimization and eigen spectrum determination
- VQE (variational quantum eigensolver) is a kind / type of VQA—that minimizes the expected energy of a molecule's Hamiltonian H using parametrized quantum circuits
- optimization problems mapped into a quantum Hamiltonian Operator H
- $H = \sum_{i,j} c_{ij} \sigma_i^2 \sigma_j^2 + \sum_i b_i \sigma_i^2$
- $\sigma_i^2 \mapsto$ pauli Z operator acting on the i th qubit
- c_{ij}, b_j are problem specific coefficients
- $E(\theta)_{VQE} = \langle \psi(\theta) | H | \psi(\theta) \rangle$
- classical optimizer adjusts parameters θ to minimize $E(\theta)$

Quantum approximate optimization Algorithm(QAOA)

28

- QAOA is a combinatorial optimization problem
- by iteratively applying the cost Hamiltonian
- In machine learning (classical)— cost function (error function) is for the entire data set, unlike loss function which for a data point.
- cost function helps to measure the extent to which the model is going wrong in estimating the relationship between $\mathcal{X}$, $\mathcal{Y}$ kind of R^2 function in regression model
- Machine learning itself is an optimization problem —using 'objective function'
- Hypothesis is measured using a 'cost function' in machine Learning
- in NN / QNN (quantum neural nets), recall— gradient descents, SGD(stochastic gradient descents) are optimization techniques only

- QAOA solves Combinatorial optimization problems by encoding the cost function H_C into a Hamiltonian and using a mixing Hamiltonian H_M to explore solution space
- $|\psi\rangle = e^{-i\beta H} e^{-i\gamma H_C} |\psi_0\rangle$
- $|\psi_0\rangle$ — — — *initialstate*, β, γ are tunable parameters
- QAOA popular applications are scheduling, route optimization, resource allocation in engineering

- $\mathcal{H}_J = \sum_{i < j} J_{ij} Z_i Z_j + \sum_i h_i Z_i$
- 2 components— (i)Applying cost Hamiltonian (ii) Mixing operator
- quantum circuit alternates between the above 2 components.
- optimization components of QAOA
- GOAL is taken care of by objective function
- input data $\leftrightarrow$ decision variables
- constraints $\leftrightarrow$ limitations on the variables

problem definition of Grover's Algorithm

31

- given a function 'f' that takes 'n' bits as input and yields a single bit as output, mathematically $f : 0, 1^n \mapsto 0, 1$
- usecase : search problems— to find some specific sequence of bit as output, within a vast collection of other sequences. i.e a function checks each sequence and returns True(1) if it matches X i.e $f(x)=1$
- the oracle and diffusion steps are approximately $\frac{\pi}{4}\sqrt{2^n}$ item diffusion operator amplifies the probability of the correct state.

single Mathematical basis of Grover's Algorithm

- the state evolution in Grover's algorithm amplifies the amplitude of the desired state.
- after $\sqrt{N}$ iterations, probability of measuring the target state approaches 1
- $|\psi\rangle = \frac{1}{\sqrt{N}} \sum_{i=0}^{N-1} |i\rangle \mapsto |x_i\rangle$
- $|x_i\rangle$ is the marked one.
- in between transformation $\mathcal{U}_w$ is applied to the state which flips the sign of the marked item
- oracle query in grover's $\mathcal{O}|x\rangle = \begin{cases} -|x\rangle & \text{--- } x = x_0 \\ |x\rangle & \text{else} \end{cases}$

in between transformation $\mathcal{U}_w$ of grover's and Diffusion operator

- $\mathcal{U}_w = \frac{1}{\sqrt{N}} \sum_{x=0}^{N-1} y_x |x\rangle$
- post 'oracle application' another transformation= grover's diffusion operator (Grover iterate($\mathcal{U}_s$)) is applied—— to perform an inversion about the average amplitude
- Grover's iterate $\mathcal{U}_s = 2 |s\rangle \langle s| - I$
- where $|s\rangle = \frac{1}{\sqrt{N}} \sum_{x=0}^{N-1} |x\rangle \mapsto$ equal superposition state and I is the identity operator.
- next is iteration stage.

Grover's algorithm in Brief

- speedup facts have already been discussed in this day1 part 2 only, so not repeating again.
- Grover's algorithm initializes the system in an equal superposition of all states
- $|\psi_0\rangle = \frac{1}{\sqrt{N}} \sum_{i=0}^{N-1} |x\rangle$
- it then iteratively applies 2 operations
- (i) Oracle query—flips the phase of the marked element.
2 stages of grover' oracle are— (a) G_1 —marking (b) G_2 diffusion
- Grover's oracle is iterated.
- Amplitude amplification—enhances the probability amplitude of the marked state through a reflection operation.

- shor's algo. uses QFT (quantum Fourier transform) to find the periodicity of a function.,
- using $f(x) = a^x \bmod N$
- where N—number to be factorized, a—randomly chosen integer coprime to N
- QFT $\mapsto$ quantum state from time domain to the frequency domain, identifying period r
- QFT mapping $|x\rangle \mapsto \frac{1}{\sqrt{r}} \sum_{i=0}^{r-1} e^{\frac{2\pi i k r}{r}} |k\rangle$
- once 'r' is determined, N can be factorized by computing $\gcd(a^{\frac{r}{2}} \pm 1, N)$

bird's eye view of terms / concepts used in shor's algorithm

strong advice : all this background is number theory of mathematic— can be ignored in first reading / exam purpose

- order—order of element 1 in $\mathbb{Z}_m$ is equal to 'm', as taking a sum of 'm' times= identity element.
- Example —consider $\mathbb{Z}_{10}$, here 2 has order '5 since $2+2+2+2+2=0 \bmod 10$ '. order of 3=10 as the smallest multiple of 3 which has remainder 0 mod 10 is $30= 10 \times 3 = 3 + 3 + 3 + 3 + 3 + 3 + 3 + 3 + 3 + 3 + 3 + 3$
- General formula is
- $\frac{LCM(m,k)}{k} = \frac{m}{GCD(m,k)}$
- Multiplicative inverse:Remainders a , b in $\mathbb{Z}_m$ are called multiplicative inverses of each other if $ab=1 \bmod m$

- in $\mathbb{Z}_{10}$ elements 3 and 7 are inverses of each other, as $3 \times 7 = 1 \pmod{10}$, 2 is not invertible as there no remainder b such that $2b = 1 \pmod{10} \mapsto$ basis of Euclid's algorithm

period finding example for shor's Algorithm

- for a sequence 's' created from a list of $r=4$ numbers repeated $m=3$ times for a list of 12 elements.
- sequence $s = \underbrace{3, 7, 2, 4}, \underbrace{3, 7, 2, 4}, \underbrace{3, 7, 2, 4}$
- 'r' is the repeated length **period** of the sequence that makes up the list, 'm'—number of repeats of that list in 's'
- considering 'r' divides 'N' case i.e. $m = \frac{N}{r}$, m—integer.
- considering a function $f(x) = x \bmod r$, then for inputs $x=0$ to $r-1 \implies$ the outputs of $f(x)$ from 0 to $r-1$ and...

value of $m \bmod 4$ for increasing value of m

inputs $x=0$ to $r-1$ $x=r$ to $2r-1$ to produce outputs of $f(x)=0$ to $r-1$,
inputs $x=r$ to $x=r$ to $2r-1$ and again

m	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
$m \bmod 4$	0	1	2	3	0	1	2	3	0	1	2	3	0	1	2

sequence of 4 elements being repeated 3 times

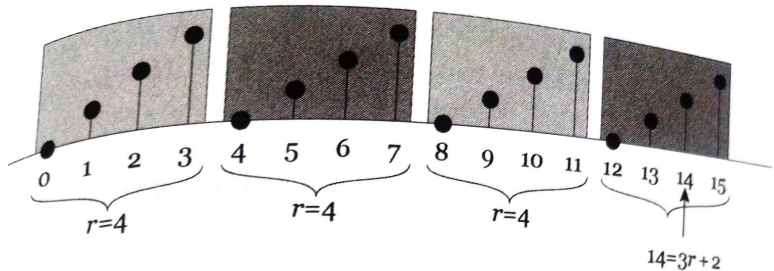


Figure: from a printed book

x	7^x	$7^x \bmod 15$
0	1	1
1	7	7
2	49	4
3	343	13
4	2,401	1
5	16,807	7
6	117,649	4
7	823,543	13
8	5,764,801	1
9	40,353,607	7
10	282,475,249	4
11	1,977,326,743	13

Figure: from a published book

shor's algo initial steps

1. **Setup:** We're given $M = 15$.
2. **Pick circuit size:** Because $M^2 = 225$, any value of $n \geq 8$ will do, as $2^8 = 256 > 225$. Let's arbitrarily pick $n = 11$.
3. **Pick a :** Arbitrarily pick $a = 7$.
4. **See if we're lucky:** Find the gcd of a and M . Because $\gcd(7, 15) = 1$, we weren't lucky, so we continue.
5. **Go quantum:** Build the quantum gate that implements the function

Figure: from a published book with thanks

shor's algorithm in 5 steps

- take a value and raise to the power $r=r^x$ and find how bigger it is, compared to $m.N$ and see $r^x - m.N > .$
quantum computer puts r^x in a superposition state in which $x=1,2,3....$ at same time
- 2nd stage—QC uses superposition to calculate ' x ', $r^x - m.N >$ where $x= 1,2,3...$ (benefit of quantum computer)
- if $r^x = m.N + 1$, then $\exists$ a condition where $r^{x+p} = m_1, N + 1$. these values maintain a frequency. such frequency can be calculated by **inversing period**, where QFT is useful

- QFT can perform destructive interference and can give one value of 'p'. if p is even, then substitute it in $r^{\frac{p}{2}} \pm 1$ to get the factors.
- if p is odd then start again from guess (random number)

worked out example— $N=15$ factorization (shor's)

on a quantum computer , procedure of factorization :

- ① take an integer x that is less than and prime to N , say $x=7$
- ② to find order ' r ' of the congruence $x^r \equiv 1 \pmod{n}$, perform the Hadamard transformation ' n ' times on the first register of the initial state $|0\rangle^{\otimes n} |1\rangle = \frac{1}{\sqrt{n}} \sum_{a=0}^{2^n-1} |a\rangle |1\rangle$, with $n=11$,
- ③ find $f(a) \equiv x^a \pmod{N}$ and put the result in 2nd register
i.e $\frac{1}{\sqrt{2^n}} \sum_{a=0}^{2^n-1} |a\rangle |x^a \pmod{N}\rangle = \frac{1}{2^n} \left(|0\rangle |1\rangle + |1\rangle |7\rangle + |2\rangle |4\rangle + |3\rangle |13\rangle + |4\rangle |1\rangle + |5\rangle |7\rangle + |6\rangle |4\rangle + \dots \right)$
- ④ measure the 2nd register to find $r=0,4,8,\dots$ satisfying $x^r \equiv 1 \pmod{N}$

shor's factorization example step 5(contd)

step 5:

from $x^{\frac{r}{2}} \pm 1 = 7^2 \pm 1 = 48, 50$

$\gcd(48, 15) = 3$ and $\gcd(50, 15) = 5$

$\implies 3$ and 5 as the prime factors of $N=15$.

DFT $\leftrightarrow$ QFT analogy

- DFT (discrete Fourier transform) in classical signal processing $\equiv$ QFT (quantum Fourier transform) in quantum computing.
- both DFT, QFT Time domain to $\mapsto$ frequency domain transformation.
- starting from DFT, i.e transformation from one vector to another
- $[x_0, x_1, \dots, x_{N-1}] \xleftrightarrow[\text{dft}]{\text{idft}} [f_0, f_1, \dots, f_{N-1}]$ bothways
- after normalization $\implies$
- $f_k = \left(\frac{1}{\sqrt{N}} \sum_{j=0}^{N-1} x_j e^{\frac{-i2\pi kj}{N}} \right)$
- putting $e^{\frac{i.2\pi}{N}} = \omega \implies$

- $f_k = \frac{1}{\sqrt{N}} \sum_{j=0}^{N-1} x_j \omega^{-kj}$
- above equation = kind of Matrix times a vector
- so, operator must be unitary
- so, QFT with 1 qubit=Hadamard only (homework)
-